Nome: João Geiger Piza

Resumo dos principais componentes de uma CNN:

Uma CNN é formada principalmente por camadas de convolução, que extraem padrões das imagens, por camadas de ativação, que introduzem não linearidade para permitir aprendizado mais complexo, por camadas de pooling, que reduzem a dimensão dos dados e ajudam a tornar a rede mais eficiente, e por camadas totalmente conectadas, que combinam as informações extraídas para realizar a classificação final. Em muitos modelos também há normalização e dropout, que ajudam a estabilizar o treinamento e reduzir o overfitting.

Descrição do experimento realizado com PyTorch:

O experimento realizado envolveu criar uma CNN com PyTorch e treiná-la usando um dataset de números escritos à mão (0 à 9) e analisar sua acurácia e seu custo por época. Após isso, várias modificações foram feitas, e o impacto destast foi analisado.

Respostas dos exercícios (Texto):

1) No código, foi definido a variável device como:

```
device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
```

E, ao instalar o modelo, utilizei o método .to(device) para movê-lo para a gpu. Além disso, antes dos feed forwards tanto no treinamento quanto no teste, movi images e labes para device também, usando o mesmo método. Após estas alterações, foi notável a redução do tempo de execução.

2) Realizei as seguintes operações:

```
train_size = int(0.9 * len(train_dataset))
val_size = len(train_dataset) - train_size

train_subset, val_subset = random_split(train_dataset,
[train_size, val_size])

train_loader = DataLoader(dataset=train_subset,
batch_size=batch_size, shuffle=True)
val_loader = DataLoader(dataset=val_subset, batch_size=batch_size,
shuffle=False)
```

Depois, dentro do loop de épocas, coloquei o loop de validação da seguinte forma:

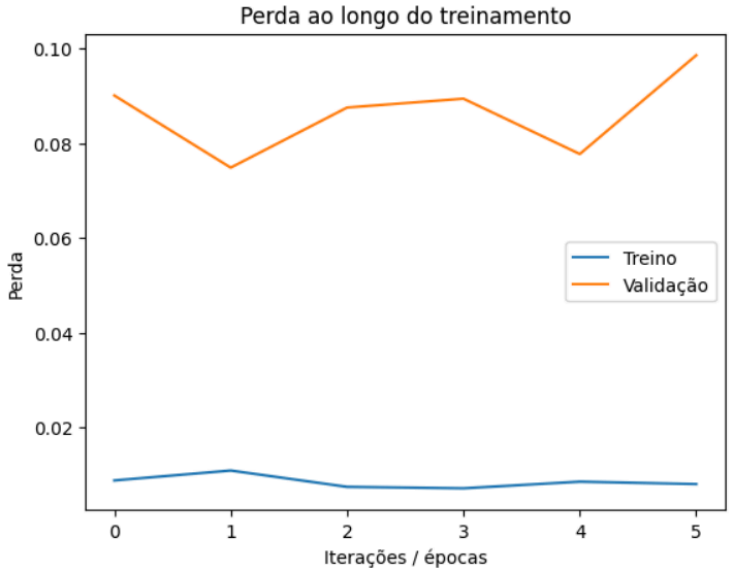
```
model.eval()
val_loss = 0
val_correct = 0
val_total = 0

with torch.no_grad():
      for images, labels in val_loader:
```

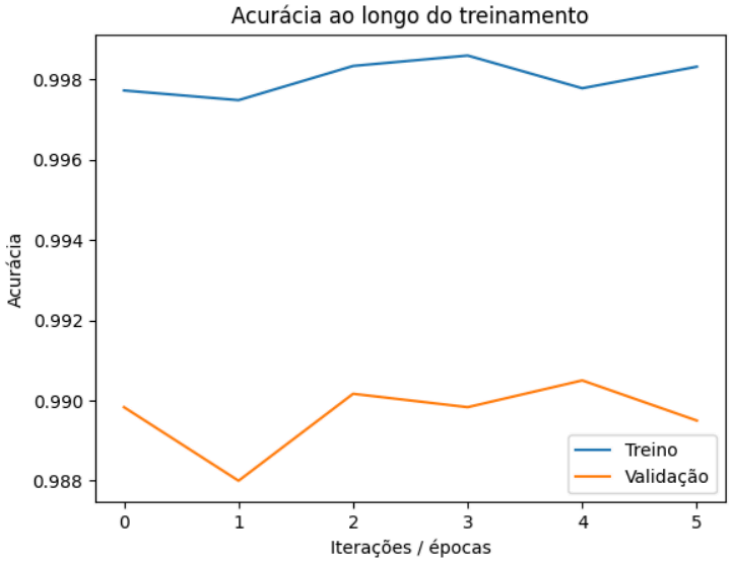
```
                images = images.to(device)
                labels = labels.to(device)

                outputs = model(images)
                loss = loss_fn(outputs, labels)

                val_loss += loss.item()

                _, predicted = torch.max(outputs.data, 1)
                val_total += labels.size(0)
                val_correct += (predicted == labels).sum().item()

    avg_val_loss = val_loss / len(val_loader)
    val_acc = val_correct / val_total
    val_loss_list.append(avg_val_loss)
    val_acc_list.append(val_acc)
    print(f"Validação - Perda: {round(avg_val_loss, 3)}, Acurácia:
{round(val_acc * 100, 3)}")
```

3) Primeiro, criei listas e modifiquei o código para coletar as informações tanto de acurácia quanto de perda por época. Então, após adicionar o seguinte trecho de código após o final do treinamento:

```
plt.figure()
plt.plot(train_loss_list, label="Treino")
plt.plot(val_loss_list, label="Validação")
plt.xlabel("Iterações / épocas")
plt.ylabel("Perda")
plt.title("Perda ao longo do treinamento")
plt.legend()
plt.show()

plt.figure()
plt.plot(train_acc_list, label="Treino")
plt.plot(val_acc_list, label="Validação")
plt.xlabel("Iterações / épocas")
plt.ylabel("Acurácia")
plt.title("Acurácia ao longo do treinamento")
plt.legend()
plt.show()
```

Obtive os seguintes gráficos:





4) Para salvar o modelo de melhor desempenho, criei uma variável best_val_acc, inicializada como 0, e coloquei dentro do loop de épocas, após o loop de validação:

```
if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), "best_model.pth")
```

Para carregar esses pesos, basta fazer algo como:

```
model2 = ConvNet().to(device)
```

```
model2.load_state_dict(torch.load("best_model.pth",
map_location=device))
```

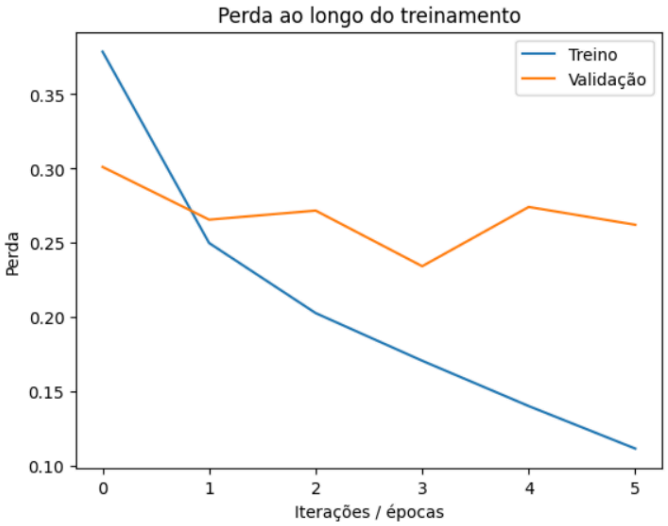
5) Durante os experimentos com o modelo, algumas alterações foram feitas na arquitetura e na etapa de treinamento para buscar um melhor desempenho. Entre as mudanças mais relevantes esteve a adoção do otimizador SGD, que mostrou uma convergência menos ágil em comparação com o Adam. Também foram feitos ajustes na taxa de aprendizado, o que afetou diretamente a estabilidade do processo de treinamento. Ao diminuir a learning rate para 0.00001, observou-se uma estabilidade na perda e na acurácia na validação do treinamento, com a perda sendo relativamente baixa (aproximadamente 0.008) e a acurácia alta (aproximadamente 99.1). Porém, ao aumentar para 0.1, nota-se alterações mais radicais na acurácia, porém o custo se mantém estável, mas o custo médio é relativamente alto (aproximadamente 2.3) e a acurácia média é muito baixa (aproximadamente 0.1).

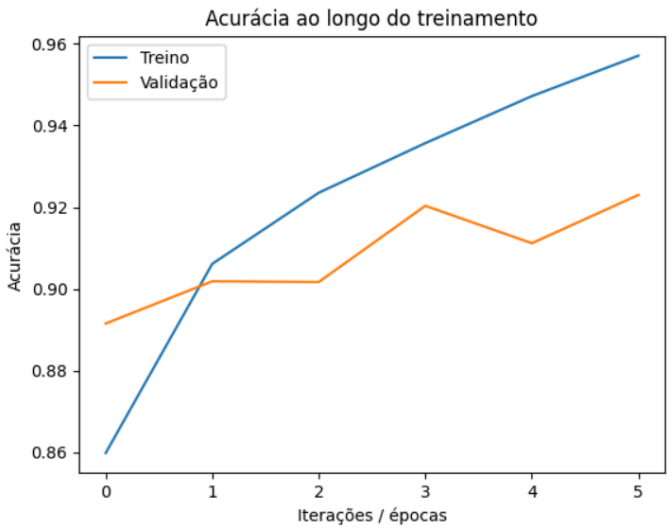
6) Os modos model.train() e model.eval() controlam o comportamento da rede neural durante o treinamento e a avaliação. No treino, camadas como dropout e batch normalization atuam de forma a favorecer o aprendizado. Já em model.eval(), o dropout é desativado e a rede passa a usar estatísticas fixas. Além disso, na avaliação, o uso de torch.no_grad() impede o cálculo de gradientes, deixando o processo mais eficiente e economizando memória.

7) Ao trocar os datasets por:

```
train_dataset = torchvision.datasets.FashionMNIST(root=root_path,
train=True, transform=trans, download=True)
test_dataset = torchvision.datasets.FashionMNIST(root=root_path,
train=False, transform=trans, download=True)
```

e treinar o modelo em cima destes, obtive os seguintes resultados:





Isso indica que o modelo proposto não é tão eficiente com esse dataset quanto com o original.

Respostas dos exercícios (Código):

```
import torchvision
import torch.nn as nn
import matplotlib.pyplot as plt
from torchvision import transforms
from torch.utils.data import DataLoader
from torch.utils.data import random_split
from torch.optim import Adam

class ConvNet(nn.Module):
    def __init__(self):
        super(ConvNet, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=5, stride=1, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2))
        self.layer2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=5, stride=1, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2))
        self.fc1 = nn.Sequential(
            nn.Linear(7 * 7 * 64, 1000),
            nn.ReLU())
        self.fc2 = nn.Linear(1000, 10)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
```

```
        out = out.reshape(out.size(0), -1)
        out = self.fc1(out)
        out = self.fc2(out)
        return out


device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# MNIST dataset
root_path = '/home/storopoli/Downloads' # mude isso no Colab se
necessário

# Pequena transformação para tensores e normalizando o tamanho
trans = transforms.Compose([transforms.ToTensor(),
transforms.Normalize((0.1307,), (0.3081,))])

# Train/Test Datasets
train_dataset = torchvision.datasets.FashionMNIST(root=root_path,
train=True, transform=trans, download=True)
test_dataset = torchvision.datasets.FashionMNIST(root=root_path,
train=False, transform=trans, download=True)

batch_size=32

train_size = int(0.9 * len(train_dataset))
val_size = len(train_dataset) - train_size

train_subset, val_subset = random_split(train_dataset, [train_size,
val_size])

train_loader = DataLoader(dataset=train_subset, batch_size=batch_size,
shuffle=True)
val_loader = DataLoader(dataset=val_subset, batch_size=batch_size,
shuffle=False)
test_loader = DataLoader(dataset=test_dataset, batch_size=batch_size,
shuffle=False)

# Hiperparâmetros
loss_fn = nn.CrossEntropyLoss()
learning_rate = 0.001
epochs = 6
```

```python
# Instânciar o Otimizador Adam
optimizer = Adam(model.parameters(), lr=learning_rate)

# Treinar o Modelo
total_step = len(train_loader) # quantos batches eu tenho

# Listas vazias
train_loss_list = []
train_acc_list = []
val_loss_list = []
val_acc_list = []

# Precisão do melhor
best_val_acc = 0

for epoch in range(epochs):
    model.train()
    train_loss = 0
    train_correct = 0
    train_total = 0
    for i, (images, labels) in enumerate(train_loader):
        # Gera a propagação (feed forward)
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)

        # Calcula a função-custo
        loss = loss_fn(outputs, labels)
        train_loss += loss.item()

        # Retro-propagação (Backprop) e a otimização com Adam
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # Acurácia
        total = labels.size(0)
        train_total += total
        _, predicted = torch.max(outputs.data, 1)
        correct = (predicted == labels).sum().item()
        train_correct += correct
```

```
        if (i + 1) % 100 == 0:
            print(f"Época [{epoch+1}/{epochs}], Step
[{i+1}/{total_step}], Custo: {round(loss.item(), 3)}, Acurácia:
{round((correct / total) * 100, 3)}")

    avg_train_loss = train_loss / len(train_loader)
    train_acc = train_correct / train_total

    train_loss_list.append(avg_train_loss)
    train_acc_list.append(train_acc)

    model.eval()
    val_loss = 0
    val_correct = 0
    val_total = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = loss_fn(outputs, labels)

            val_loss += loss.item()

            _, predicted = torch.max(outputs.data, 1)
            val_total += labels.size(0)
            val_correct += (predicted == labels).sum().item()

    avg_val_loss = val_loss / len(val_loader)
    val_acc = val_correct / val_total
    val_loss_list.append(avg_val_loss)
    val_acc_list.append(val_acc)

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), "best_model.pth")

    print(f"Validação - Perda: {round(avg_val_loss, 3)}, Acurácia:
{round(val_acc * 100, 3)}")
```

```
# Como carregar as infos em uma nova instância:
# model2 = ConvNet().to(device)
# model2.load_state_dict(torch.load("best_model.pth",
map_location=device))
# model2.eval()

plt.figure()
plt.plot(train_loss_list, label="Treino")
plt.plot(val_loss_list, label="Validação")
plt.xlabel("Iterações / épocas")
plt.ylabel("Perda")
plt.title("Perda ao longo do treinamento")
plt.legend()
plt.show()

plt.figure()
plt.plot(train_acc_list, label="Treino")
plt.plot(val_acc_list, label="Validação")
plt.xlabel("Iterações / épocas")
plt.ylabel("Acurácia")
plt.title("Acurácia ao longo do treinamento")
plt.legend()
plt.show()
model.eval() # coloca o modelo em modo de avaliação (sem calcular
gradientes)

with torch.no_grad():
    correct = 0
    total = 0
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)
        # Feed-forward com as imagens de teste
        outputs = model(images)

        # gera predições usando a função max()
        _, predicted = torch.max(outputs.data, 1)

        # Acumula total e corretas
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

```
    print(f"Acurácia do Modelo em 10k imagens de teste: {round((correct
/ total) * 100, 3)}")
```